Roshaan Ameer
03/27/2026

Firewall

Overview

A stateful firewall and port forwarding for the services operating within the www container are how I want to secure the virtual machine. Since it seems like the safest and most sensible way to manage access, I want the virtual machine to block incoming traffic unless I have explicitly permitted it. I will let SSH, HTTP, and HTTPS while blocking everything else because I just require a few services to be accessible from the outside. This, in my opinion, makes the system easier to manage and reduces the possibility of revealing stuff that shouldn't be open.

I like to begin from the secure side rather than trying to repair issues after leaving things overly open, which is why I'm employing a default-deny strategy for the firewall. Since I can specify exactly what should be allowed in and keep everything else closed, I think this method is more practical and clean. Additionally, I selected a stateful firewall because it can monitor active connections, allowing regular return traffic to flow through without the need for needless additional rules. In terms of port forwarding, I want the www container to operate the actual services while the virtual machine serves as the primary access point. I will forward port 2222 on the VM to port 22 on the container, and I will also forward ports 80 and 443 on the VM to the same ports on the www container so web traffic reaches the right place.

I used tools like hostname, whoami, ip -br addr, ip route, lxc list, and uname -a to verify the system and network details before making any modifications. In order to fully comprehend the setup before configuring anything, I used these commands to verify the machine name, current user, network interfaces, routing details, running containers, and system data. To properly route the traffic to the container, I will then apply the firewall rules, activate IP forwarding, and add port forwarding rules. After the setup is complete, I'll test each service to make sure everything functions as I had anticipated, and I'll attach screenshots to properly document the outcome.

```
File  Actions  Edit  View  Help
roshaanameer@roshaan-cyberlion: ~
roshaanameer@roshaan-cyberlion:~$ whoami
roshaanameer
roshaanameer@roshaan-cyberlion:~$ hostname
roshaan-cyberlion
roshaanameer@roshaan-cyberlion:~$ ip -br addr
lo               UNKNOWN        127.0.0.1/8 ::1/128
enp0s3           UP             10.0.2.15/24 metric 100 fe80::a00:27ff:fe00:9fdc/64
lxdbr0           UP             172.16.31.1/24
veth1bfd0d69@if4 UP
veth5cbf3095@if36 UP
vethcf417b23@if38 UP
veth13e80f28@if40 UP
vethcb33e5b4@if42 UP
vethfb9c0bf6@if46 UP
veth5e40dcda@if48 UP
vethaac874d7@if50 UP
roshaanameer@roshaan-cyberlion:~$ ip route
default via 10.0.2.2 dev enp0s3 proto dhcp src 10.0.2.15 metric 100
10.0.2.0/24 dev enp0s3 proto kernel scope link src 10.0.2.15 metric 100
10.0.2.2 dev enp0s3 proto dhcp scope link src 10.0.2.15 metric 100
10.23.0.20 via 10.0.2.2 dev enp0s3 proto dhcp src 10.0.2.15 metric 100
10.23.0.30 via 10.0.2.2 dev enp0s3 proto dhcp src 10.0.2.15 metric 100
172.16.31.0/24 dev lxdbr0 proto kernel scope link src 172.16.31.1
roshaanameer@roshaan-cyberlion:~$  lxc list
+---------+---------+----------------------+------+-----------+-----------+
|  NAME   |  STATE  |         IPV4         | IPV6 |   TYPE    | SNAPSHOTS |
+---------+---------+----------------------+------+-----------+-----------+
| c0      | RUNNING | 172.16.31.254 (eth0) |      | CONTAINER | 0         |
+---------+---------+----------------------+------+-----------+-----------+
| client  | RUNNING | 172.16.31.15 (eth0)  |      | CONTAINER | 0         |
+---------+---------+----------------------+------+-----------+-----------+
| client2 | RUNNING | 172.16.31.16 (eth0)  |      | CONTAINER | 0         |
+---------+---------+----------------------+------+-----------+-----------+
| krb     | RUNNING | 172.16.31.13 (eth0)  |      | CONTAINER | 0         |
+---------+---------+----------------------+------+-----------+-----------+
```

```
File  Actions  Edit  View  Help
roshaanameer@roshaan-cyberlion: ~
veth5e40dcda@if48 UP
vethaac874d7@if50 UP
roshaanameer@roshaan-cyberlion:~$ ip route
default via 10.0.2.2 dev enp0s3 proto dhcp src 10.0.2.15 metric 100
10.0.2.0/24 dev enp0s3 proto kernel scope link src 10.0.2.15 metric 100
10.0.2.2 dev enp0s3 proto dhcp scope link src 10.0.2.15 metric 100
10.23.0.20 via 10.0.2.2 dev enp0s3 proto dhcp src 10.0.2.15 metric 100
10.23.0.30 via 10.0.2.2 dev enp0s3 proto dhcp src 10.0.2.15 metric 100
172.16.31.0/24 dev lxdbr0 proto kernel scope link src 172.16.31.1
roshaanameer@roshaan-cyberlion:~$  lxc list
+---------+---------+----------------------+------+-----------+-----------+
|  NAME   |  STATE  |         IPV4         | IPV6 |   TYPE    | SNAPSHOTS |
+---------+---------+----------------------+------+-----------+-----------+
| c0      | RUNNING | 172.16.31.254 (eth0) |      | CONTAINER | 0         |
+---------+---------+----------------------+------+-----------+-----------+
| client  | RUNNING | 172.16.31.15 (eth0)  |      | CONTAINER | 0         |
+---------+---------+----------------------+------+-----------+-----------+
| client2 | RUNNING | 172.16.31.16 (eth0)  |      | CONTAINER | 0         |
+---------+---------+----------------------+------+-----------+-----------+
| krb     | RUNNING | 172.16.31.13 (eth0)  |      | CONTAINER | 0         |
+---------+---------+----------------------+------+-----------+-----------+
| ldap    | RUNNING | 172.16.31.12 (eth0)  |      | CONTAINER | 0         |
+---------+---------+----------------------+------+-----------+-----------+
| ns1     | RUNNING | 172.16.31.10 (eth0)  |      | CONTAINER | 0         |
+---------+---------+----------------------+------+-----------+-----------+
| ns2     | RUNNING | 172.16.31.11 (eth0)  |      | CONTAINER | 0         |
+---------+---------+----------------------+------+-----------+-----------+
| www     | RUNNING | 172.16.31.17 (eth0)  |      | CONTAINER | 0         |
+---------+---------+----------------------+------+-----------+-----------+
roshaanameer@roshaan-cyberlion:~$ uname -a
Linux roshaan-cyberlion 6.8.0-106-generic #106-Ubuntu SMP PREEMPT_DYNAMIC Fri Mar  6 07:58:08 UTC 2026 x86_64 x86
_64 x86_64 GNU/Linux
roshaanameer@roshaan-cyberlion:~$
```

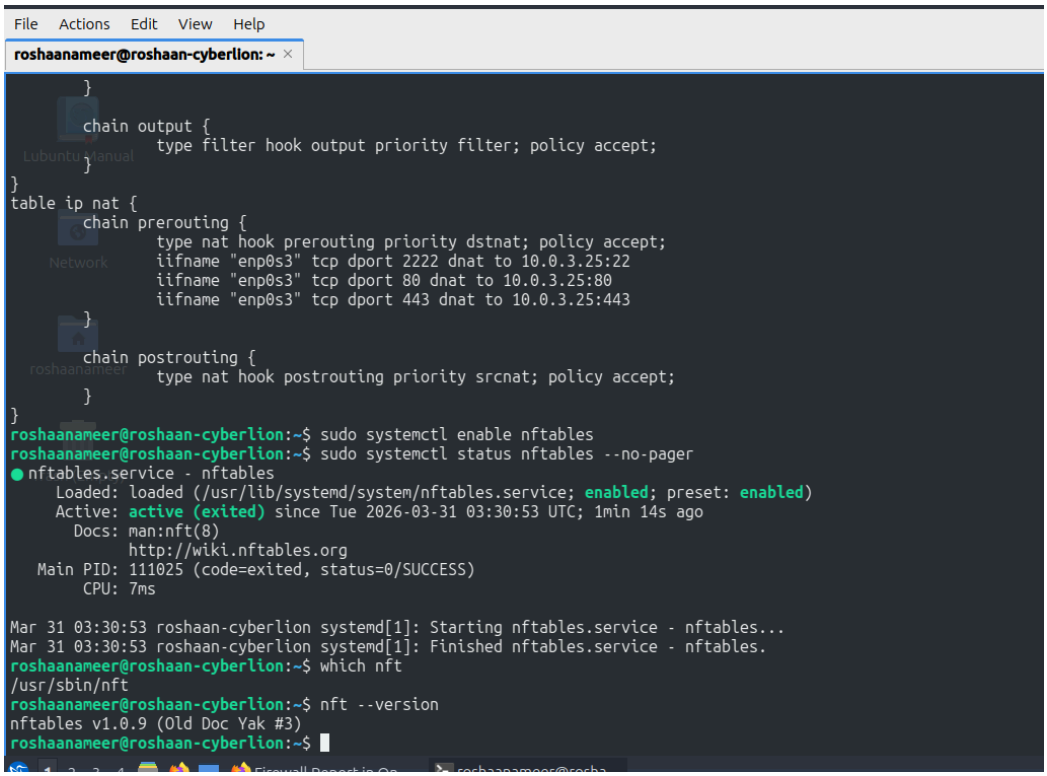
Firewall Configuration Plan

Because I believe it makes the most sense for this type of setup, I'm going to set up the firewall on the virtual machine using a stateful filtering approach. I don't want the firewall to merely consider IP addresses and port numbers, treating every packet as a distinct entity. When traffic is a part of an actual, authorized connection, I want it to recognize that. In this manner, the response traffic can return properly if I permit legitimate traffic, such as SSH or web traffic, and I won't

need to open more ports. In my opinion, this makes the firewall easier to maintain and more secure than a stateless configuration.

For inbound traffic, I intend to use a default-deny rule, which implies that unless I have a special rule that permits it, anything entering the virtual machine will be blocked. This strategy appeals to me since it begins with safety rather than leaving the system too open. I would prefer to keep the machine locked by default and only permit what is truly necessary, as opposed to permitting a lot of traffic at first and then attempting to reduce it later. I will permit SSH on port 22, HTTP on port 80, and HTTPS on port 443 because I only need these three services to be accessible for my configuration. I will also allow established and related traffic because valid connections still need to send reply traffic back and forth in a normal way. Any traffic that is invalid, unexpected, or not clearly matched by one of my rules will be dropped.

This firewall design appeals to me since it maintains simplicity while feeling powerful and purposeful. By blocking everything unless I decide to let it and opening only the ports that are truly required, I am establishing a distinct barrier around the virtual computer. The firewall can let acceptable return traffic without making the rules more permissive than necessary since it monitors connection status. This keeps the system more secure overall while providing me with a regulated setup for management and online access. It also aligns with the general security strategy I want to use, which is to avoid exposing more of the system than is necessary in favor of keeping things small, transparent, and simpler to defend.



Port Forwarding Plan

To transport particular incoming traffic from the virtual machine to the services operating inside the www container, I'm going to use port forwarding. I have to do this since the actual SSH and web services are operating inside the container, but the virtual machine (VM) is the primary system that external traffic can access first. I'm utilizing the virtual machine (VM) as the point of entry and only forwarding the traffic that needs to reach www because I don't want to expose the container directly. This configuration allows the container to deliver the real services behind the scenes while maintaining control over external access, which is why I prefer it.

My forwarding strategy adheres to the setup's necessary ports. In order for SSH to access the container via the VM, I will route traffic that arrives on port 2222 of the VM to port 22 on the www container. Additionally, I will forward traffic arriving on port 80 of the virtual machine to port 80 on the container and traffic arriving on port 443 of the virtual machine to port 443 on the same container. Since the container still manages the actual service while the virtual machine functions as the front entrance, I think this is a clear method to handle access. This gives me a single primary location to manage incoming traffic while maintaining an easier-to-understand layout.

In order for that to function, I must first determine the IP address of the www container and enable IP forwarding on the virtual machine (VM) so that packets can pass through it correctly. In order to translate and send the traffic to the appropriate location inside the container, I will then build NAT port forwarding rules in nftables. This strategy, in my opinion, works well for the setup since it allows me to manage port forwarding and firewall filtering in one location and establishes a distinct division between the internal container that executes the services and the virtual machine that is visible to the public.

```
File  Actions  Edit  View  Help
roshaanameer@roshaan-cyberlion: ~
net.ipv4.ip_forward = 1
roshaanameer@roshaan-cyberlion:~$ sudo nft list ruleset
table inet filter {
        chain input {
                type filter hook input priority filter; policy drop;
                iif "lo" accept
                ct state established,related accept
                ct state invalid drop
                tcp dport 22 accept
                tcp dport 80 accept
                tcp dport 443 accept
        }

        chain forward {
                type filter hook forward priority filter; policy drop;
                ct state established,related accept
                iifname "enp0s3" ip daddr 10.0.3.25 tcp dport 22 accept
                iifname "enp0s3" ip daddr 10.0.3.25 tcp dport 80 accept
                iifname "enp0s3" ip daddr 10.0.3.25 tcp dport 443 accept
        }
        chain output {
                type filter hook output priority filter; policy accept;
        }
}
table ip nat {
        chain prerouting {
                type nat hook prerouting priority dstnat; policy accept;
                iifname "enp0s3" tcp dport 2222 dnat to 10.0.3.25:22
                iifname "enp0s3" tcp dport 80 dnat to 10.0.3.25:80
                iifname "enp0s3" tcp dport 443 dnat to 10.0.3.25:443
        }

        chain postrouting {
                type nat hook postrouting priority srcnat; policy accept;
```

Tools and Technology

Since nftables is the typical contemporary framework for packet filtering and NAT on Linux, I shall utilize it as the primary firewall technology. Because it allows both stateful firewall rules and port forwarding inside a single ruleset, it is an excellent fit for this design. This enables the virtual machine to be secured and traffic being sent to the www container to be managed in an orderly and transparent manner. Documenting and maintaining the configuration is made easier by using a single framework for both forwarding and filtering.

To enable the necessary services to operate independently of the virtual machine itself, I will host the www container using LXD. With the container running the internal services and the virtual machine serving as the primary access point, this division enhances organization and reflects a more structured system design. Additionally, by maintaining a logical separation between the containerized services and the host firewall settings, it simplifies environment management.

I will examine the system and confirm the settings using common Linux networking tools like ip, ss, and sysctl in addition to nftables and LXD. These tools will assist me in locating network interfaces, verifying routing details, examining listening ports, and turning on packet forwarding when necessary. These technologies work together to offer everything required for the setup: LXD provides the service container, nftables manages filtering and forwarding, and Linux utilities help verification and debugging.

```
File  Actions  Edit  View  Help
roshaanameer@roshaan-cyberlion: ~
        }
}
roshaanameer@roshaan-cyberlion:~$ which nft
/usr/sbin/nft
roshaanameer@roshaan-cyberlion:~$ nft --version
nftables v1.0.9 (Old Doc Yak #3)
roshaanameer@roshaan-cyberlion:~$ systemctl status nftables --no-pager
● nftables.service - nftables
     Loaded: loaded (/usr/lib/systemd/system/nftables.service; enabled; preset: enabled)
     Active: active (exited) since Tue 2026-03-31 03:30:53 UTC; 9min ago
       Docs: man:nft(8)
             http://wiki.nftables.org
   Main PID: 111025 (code=exited, status=0/SUCCESS)
        CPU: 7ms

Mar 31 03:30:53 roshaan-cyberlion systemd[1]: Starting nftables.service - nftables...
Mar 31 03:30:53 roshaan-cyberlion systemd[1]: Finished nftables.service - nftables.
roshaanameer@roshaan-cyberlion:~$ which lxc
/usr/sbin/lxc
roshaanameer@roshaan-cyberlion:~$ lxc version
Client version: 5.21.4 LTS
Server version: 5.21.4 LTS
roshaanameer@roshaan-cyberlion:~$ ip -br addr
lo               UNKNOWN        127.0.0.1/8 ::1/128
enp0s3           UP             10.0.2.15/24 metric 100 fe80::a00:27ff:fe00:9fdc/64
lxdbr0           UP             172.16.31.1/24
veth1bfd0d69@if4 UP
veth5cbf3095@if36 UP
vethcf417b23@if38 UP
veth13e80f28@if40 UP
vethcb33e5b4@if42 UP
vethfb9c0bf6@if46 UP
veth5e40dcda@if48 UP
vethaac874d7@if50 UP
roshaanameer@roshaan-cyberlion:~$ ip route
```

```
File  Actions  Edit  View  Help
roshaanameer@roshaan-cyberlion: ~
vethcb33e5b4@if42 UP
vethfb9c0bf6@if46 UP
veth5e40dcda@if48 UP
vethaac874d7@if50 UP
roshaanameer@roshaan-cyberlion:~$ ip route
default via 10.0.2.2 dev enp0s3 proto dhcp src 10.0.2.15 metric 100
10.0.2.0/24 dev enp0s3 proto kernel scope link src 10.0.2.15 metric 100
10.0.2.2 dev enp0s3 proto dhcp scope link src 10.0.2.15 metric 100
10.23.0.20 via 10.0.2.2 dev enp0s3 proto dhcp src 10.0.2.15 metric 100
10.23.0.30 via 10.0.2.2 dev enp0s3 proto dhcp src 10.0.2.15 metric 100
172.16.31.0/24 dev lxdbr0 proto kernel scope link src 172.16.31.1
roshaanameer@roshaan-cyberlion:~$ sysctl net.ipv4.ip_forward
net.ipv4.ip_forward = 1
roshaanameer@roshaan-cyberlion:~$
roshaanameer@roshaan-cyberlion:~$ ss -tuln
Netid  State      Recv-Q     Send-Q         Local Address:Port          Peer Address:Port      Process
udp    UNCONN     0          0                   0.0.0.0:41274              0.0.0.0:*
udp    UNCONN     0          0                   0.0.0.0:41616              0.0.0.0:*
udp    UNCONN     0          0                   0.0.0.0:5353               0.0.0.0:*
udp    UNCONN     0          0                   0.0.0.0:36210              0.0.0.0:*
udp    UNCONN     0          0                 172.16.31.1:53               0.0.0.0:*
udp    UNCONN     0          0                  127.0.0.54:53               0.0.0.0:*
udp    UNCONN     0          0                127.0.0.53%lo:53              0.0.0.0:*
udp    UNCONN     0          0                0.0.0.0%lxdbr0:67             0.0.0.0:*
udp    UNCONN     0          0              10.0.2.15%enp0s3:68             0.0.0.0:*
udp    UNCONN     0          0                         [::]:57610              [::]:*
udp    UNCONN     0          0                         [::]:5353               [::]:*
tcp    LISTEN     0          4096             127.0.0.53%lo:53              0.0.0.0:*
tcp    LISTEN     0          4096                 127.0.0.1:631             0.0.0.0:*
tcp    LISTEN     0          4096                127.0.0.54:53              0.0.0.0:*
tcp    LISTEN     0          32                  172.16.31.1:53              0.0.0.0:*
tcp    LISTEN     0          4096                     [::1]:631              [::]:*
tcp    LISTEN     0          4096                         *:443                 *:*
tcp    LISTEN     0          4096                         *:80                  *:*
roshaanameer@roshaan-cyberlion:~$
```

Expected Outcomes

The virtual machine should function as a secure gateway for the www container once the firewall and port forwarding configuration are complete. Unless a special rule permits it, the firewall should by default block all incoming traffic. Only the necessary services, SSH on port 22, HTTP

on port 80, and HTTPS on port 443, should therefore continue to be reachable from outside the virtual machine. Any more unwanted incoming traffic must to be turned away. This adheres to the standard firewall principle of default deny, which exposes just essential services while blocking everything else.

It is anticipated that the port forwarding setup will enable the www container to be accessed via the virtual machine (VM) without directly exposing the container. To enable SSH access to the container via the host VM, a connection to port 2222 on the VM should be routed to port 22 on the www container. Similarly, the equivalent web service ports on the www container should receive traffic sent to ports 80 and 443 on the virtual machine. This means that while the services are still operating inside the container, the virtual machine (VM) continues to serve as the public entry point.

The ideal result will be observable and quantifiable from the standpoint of testing. The VM should be accessible via SSH on port 22. Requests to the VM via HTTP and HTTPS should also function. The www container should be reached using SSH access on port 2222. Concurrently, efforts to connect to unapproved ports should be unsuccessful, indicating that the firewall is implementing the desired limitations. When taken as a whole, these findings will demonstrate that the assignment requirements have been fulfilled, that forwarding is operating as intended, and that the virtual machine is securely protected.

```
roshaanameer@roshaan-cyberlion:~$ ss -tuln
Netid       State      Recv-Q     Send-Q                 Local Address:Port              Peer Address:Port      Process
udp         UNCONN     0          0                            0.0.0.0:41274                  0.0.0.0:*
udp         UNCONN     0          0                            0.0.0.0:5353                   0.0.0.0:*
udp         UNCONN     0          0                            0.0.0.0:38214                  0.0.0.0:*
udp         UNCONN     0          0                            0.0.0.0:42515                  0.0.0.0:*
udp         UNCONN     0          0                            0.0.0.0:54896                  0.0.0.0:*
udp         UNCONN     0          0                            0.0.0.0:55857                  0.0.0.0:*
udp         UNCONN     0          0                            0.0.0.0:36720                  0.0.0.0:*
udp         UNCONN     0          0                       172.16.31.1:53                      0.0.0.0:*
udp         UNCONN     0          0                        127.0.0.54:53                      0.0.0.0:*
udp         UNCONN     0          0                       127.0.0.53%lo:53                    0.0.0.0:*
udp         UNCONN     0          0                    0.0.0.0%lxdbr0:67                      0.0.0.0:*
udp         UNCONN     0          0                  10.0.2.15%enp0s3:68                      0.0.0.0:*
udp         UNCONN     0          0                             [::]:57610                     [::]:*
udp         UNCONN     0          0                             [::]:5353                      [::]:*
tcp         LISTEN     0          4096                    127.0.0.53%lo:53                     0.0.0.0:*
tcp         LISTEN     0          4096                        127.0.0.1:631                    0.0.0.0:*
tcp         LISTEN     0          4096                       127.0.0.54:53                     0.0.0.0:*
tcp         LISTEN     0          32                      172.16.31.1:53                       0.0.0.0:*
tcp         LISTEN     0          4096                          [::1]:631                      [::]:*
tcp         LISTEN     0          4096                              *:443                         *:*
tcp         LISTEN     0          4096                              *:80                          *:*
roshaanameer@roshaan-cyberlion:~$ curl -I http://10.0.2.15
HTTP/1.1 301 Moved Permanently
Server: nginx/1.24.0 (Ubuntu)
Date: Tue, 31 Mar 2026 03:55:02 GMT
Content-Type: text/html
Content-Length: 178
Connection: keep-alive
Location: https://10.0.2.15/
```

File Actions Edit View Help

roshaanameer@roshaan-cyberlion: ~

```
     Active: inactive (dead)
TriggeredBy: ● ssh.socket
       Docs: man:sshd(8)
             man:sshd_config(5)
roshaanameer@roshaan-cyberlion:~$ lxc exec www -- systemctl enable ssh
Synchronizing state of ssh.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable ssh
Created symlink /etc/systemd/system/sshd.service → /usr/lib/systemd/system/ssh.service.
Created symlink /etc/systemd/system/multi-user.target.wants/ssh.service → /usr/lib/systemd/system/ssh.service.
roshaanameer@roshaan-cyberlion:~$ lxc exec www -- systemctl restart ssh
roshaanameer@roshaan-cyberlion:~$ lxc exec www -- ss -tuln | grep ':22'
tcp   LISTEN 0      4096             0.0.0.0:22         0.0.0.0:*
tcp   LISTEN 0      4096                [::]:22            [::]:*
roshaanameer@roshaan-cyberlion:~$ lxc exec www -- ss -tuln | grep ':443'
tcp   LISTEN 0      511              0.0.0.0:443        0.0.0.0:*
tcp   LISTEN 0      511                 [::]:443           [::]:*
roshaanameer@roshaan-cyberlion:~$ lxc exec www -- systemctl status nginx --no-pager
● nginx.service - A high performance web server and a reverse proxy server
     Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: enabled)
     Active: active (running) since Tue 2026-03-31 02:41:17 UTC; 1h 16min ago
       Docs: man:nginx(8)
   Main PID: 4878 (nginx)
      Tasks: 3 (limit: 5648)
     Memory: 6.6M (peak: 6.9M swap: 2.3M swap peak: 2.4M)
        CPU: 61ms
     CGroup: /system.slice/nginx.service
             ├─4878 "nginx: master process /usr/sbin/nginx -g daemon on; master_process on;"
             ├─4879 "nginx: worker process"
             └─4880 "nginx: worker process"

Mar 29 05:21:29 www systemd[1]: Starting nginx.service - A high performance web server and a reverse proxy server...
Mar 31 02:41:17 www systemd[1]: Started nginx.service - A high performance web server and a reverse proxy server.
roshaanameer@roshaan-cyberlion:~$ sudo cat /etc/nftables.conf
#!/usr/sbin/nft -f
flush ruleset
```